- Apple-inspired premium UI/UX।
- SpongeBob-এর original cartoon universe-এর মতো environment।
- Smooth animation এবং fluid gameplay।
- Mobile + Desktop দুই জায়গাতেই playable।
- Proper game mechanics, scoring, levels, power-ups।
- Vercel/Netlify deployment-ready।
- শুধু সুন্দর UI বানিয়ে থেমে যাওয়া যাবে না, সম্পূর্ণ playable game বানাতে হবে।

🧽 PROJECT: SPONGEBOB — JELLYFISH CATCHER

নিচের পুরো prompt-টা কপি করে Google AI Studio-তে দে।

MASTER PROJECT PROMPT

SPONGEBOB: JELLYFISH CATCHER

A Premium, Playable, Cross-Platform Underwater Arcade Game

You are an elite game developer, creative director, UI/UX designer, animation engineer, and frontend architect.

Your task is to design and develop a complete, fully playable, polished browser-based arcade game called:

SPONGEBOB: JELLYFISH CATCHER

This is a SpongeBob-inspired fan-made arcade game set in the underwater world of Bikini Bottom.

The goal is to create a charming, nostalgic, visually beautiful, highly polished, and incredibly smooth gaming experience.

This must NOT look or feel like a generic AI-generated website.

Think of the visual quality, attention to detail, animation philosophy, interaction design, and polish found in premium Apple products, combined with the colorful personality and playful atmosphere of SpongeBob SquarePants.

The final product should feel like a small premium indie game that people genuinely enjoy playing.

IMPORTANT:

Do not just create a landing page or UI prototype.

Build the actual working game, including its gameplay engine, controls, collision detection, animations, scoring, levels, power-ups, sound effects, game states, and responsive layouts.

---

1. CORE CREATIVE DIRECTION

The entire experience should follow three creative principles:

1. Apple-inspired premium design.
2. Authentic SpongeBob cartoon atmosphere.
3. Smooth, responsive, addictive arcade gameplay.

Visual identity

Imagine combining:

- Apple’s minimalist design philosophy.
- A colorful underwater cartoon environment.
- Premium indie game aesthetics.
- Soft lighting and beautiful underwater visual effects.
- Smooth character animations.
- Playful SpongeBob humor.

The interface should be clean, beautiful, and intuitive.

Avoid:

- Generic gaming website templates.
- Excessive gradients.
- Unnecessary glassmorphism.
- Overloaded interfaces.
- Cheap-looking buttons.
- Random UI elements.
- Excessive animations that hurt performance.

Every UI element should have a purpose.

Use beautiful typography, carefully balanced spacing, rounded corners, subtle shadows, and delightful micro-interactions.

The game should feel polished rather than unnecessarily complicated.

---

2. SPONGEBOB UNIVERSE & ENVIRONMENT

The game takes place in the underwater world of Bikini Bottom.

The environment should capture the recognizable atmosphere of the SpongeBob SquarePants cartoon.

Use recognizable locations and characters from the SpongeBob universe where appropriate.

Main environments

A. Jellyfish Fields

This is the primary gameplay environment.

Include:

- Beautiful underwater blue sky/water atmosphere.
- Green underwater hills.
- Colorful coral formations.
- Sea flowers.
- Floating bubbles.
- Jellyfish floating around naturally.
- Sand particles.
- Background plants.
- Subtle underwater light rays.

The environment should feel alive.

B. Bikini Bottom

Use Bikini Bottom as the main menu background and a location for future expansion.

Include recognizable underwater buildings, colorful houses, roads, and cartoon scenery.

C. SpongeBob’s Pineapple House

Use this location for the character introduction or a small loading-screen scene.

D. Patrick’s Rock

Include Patrick’s iconic rock as a recognizable environmental detail.

E. Other recognizable locations

Where appropriate, use recognizable SpongeBob-inspired environmental details from:

- Jellyfish Fields
- Bikini Bottom
- SpongeBob’s neighborhood
- Underwater pathways

IMPORTANT ASSET INSTRUCTION:

Use properly licensed or user-provided SpongeBob artwork and assets where available.

If official assets are unavailable, create original fan-made cartoon-style illustrations that capture the recognizable visual atmosphere without copying episode screenshots or extracting copyrighted artwork from websites.

Do not use random unrelated stock images.

All visual assets must maintain a consistent art direction.

---

3. MAIN MENU — PREMIUM EXPERIENCE

The main menu is extremely important.

It should immediately communicate the personality of the game.

Main menu layout

Full-screen underwater environment.

The background should feature:

- Jellyfish Fields.
- Slowly floating jellyfish.
- Floating bubbles.
- Moving sea plants.
- Underwater particles.
- Subtle ambient lighting.

SpongeBob should appear prominently in the scene holding his jellyfishing net.

The character should have subtle idle animations:

- Blinking.
- Breathing.
- Small body movements.
- Natural net movement.

Main title

Display:

SPONGEBOB

JELLYFISH CATCHER

Use playful, cartoon-inspired typography.

Main buttons

- PLAY
- HOW TO PLAY
- SETTINGS
- ABOUT

Buttons should have:

- Smooth hover animations.
- Subtle press feedback.
- Beautiful transitions.
- Responsive touch interactions.

Additional UI

Display:

- Best Score.
- Total Jellyfish Caught.
- Current Player Level.

Keep the interface minimal.

Do not overcrowd the screen.

Main menu animation

When the user opens the game:

1. Underwater background fades in.
2. Bubbles slowly float upward.
3. Jellyfish move naturally.
4. SpongeBob enters with a playful animation.
5. Game title appears smoothly.
6. Buttons animate into position.

The entire sequence should feel cinematic but quick.

Do not make users wait unnecessarily.

---

4. CORE GAMEPLAY

The main gameplay is a jellyfish-catching arcade experience.

The player controls SpongeBob inside Jellyfish Fields.

The objective is to catch as many jellyfish as possible while avoiding dangerous jellyfish and environmental obstacles.

The gameplay must feel responsive, satisfying, and easy to understand.

Player character

SpongeBob SquarePants.

The character should:

- Move smoothly.
- Have proper idle animation.
- Have walking/swimming animation.
- Have a catching animation.
- Have a shocked reaction animation.
- Have a celebration animation.

Character movement must not feel robotic.

The jellyfishing net should visually follow the character’s movement.

When SpongeBob catches a jellyfish, trigger a satisfying catching animation.

Movement system

Implement smooth movement with acceleration and deceleration.

Avoid instant, unnatural position changes.

Use a consistent game loop with delta-time-based movement so the gameplay behaves consistently across different devices and refresh rates.

Add proper collision detection.

The character must remain inside the playable environment.

---

5. GAME CONTROLS

The game must support BOTH desktop and mobile devices.

This is a critical requirement.

Desktop controls

Support:

- Arrow Left: Move left.
- Arrow Right: Move right.
- Arrow Up: Move up.
- Arrow Down: Move down.

Also support:

- WASD movement.
- Spacebar: Catch jellyfish / use net.

If appropriate, allow mouse interaction for catching.

Mobile controls

The game must be fully playable on touchscreens.

Implement a beautiful virtual joystick or directional touch controller.

Include:

- Left movement.
- Right movement.
- Up movement.
- Down movement.
- A large dedicated catch button.

The controls should be positioned near the bottom corners of the screen.

Design them specifically for mobile gaming.

Requirements:

- Large touch targets.
- Comfortable thumb positioning.
- No accidental page scrolling.
- No unwanted zooming.
- Smooth touch response.
- Multi-touch support.
- No input lag.

Use pointer events where appropriate to support touch, mouse, and stylus interactions.

The game should work properly on iPhone, Android phones, tablets, laptops, and desktop computers.

IMPORTANT:

Do not simply scale down the desktop controls for mobile.

Create a dedicated mobile gameplay layout.

---

6. JELLYFISH TYPES & SCORING SYSTEM

Create different jellyfish types with unique appearances, behaviors, and scores.

1. Normal Pink Jellyfish

- Score: 10 points.
- Normal movement speed.

- Most common jellyfish.
- Easy to catch.

2. Blue Jellyfish

- Score: 20 points.
- Moves slightly faster.
- Occasionally changes direction.

3. Golden Jellyfish

- Score: 50 points.
- Rare spawn.
- Glowing golden appearance.
- Special catch animation.
- Bonus sound effect.

4. Electric Jellyfish

- Dangerous.
- Must be avoided.
- Touching one causes SpongeBob to receive an electric shock.
- Reduces one life or a portion of the player’s health.
- Triggers a funny shocked animation.

5. Rainbow Jellyfish

- Extremely rare.
- Score: 100 points.
- Appears during special events.
- Gives an additional bonus.

Each jellyfish should have:

- Independent movement.
- Different movement speeds.
- Natural floating animation.
- Randomized movement patterns.
- Individual spawn logic.
- Collision detection.
- Catching effects.

Jellyfish should never look static.

They should feel alive.

---

7. CATCHING MECHANICS

The catching system should be satisfying.

When the player presses the catch button:

1. SpongeBob performs a net-catching animation.
2. The net moves through a defined catching area.
3. Nearby catchable jellyfish inside the hitbox are collected.

4. Score increases.
5. A floating score indicator appears.
6. A small particle effect plays.
7. A satisfying sound effect is triggered.

The catching system must be fair.

Do not make catching unnecessarily difficult.

Make the timing and hitbox mechanics consistent.

Add a small visual indicator when a jellyfish is within catching range.

---

8. POWER-UP SYSTEM

Introduce collectible power-ups.

These should spawn occasionally throughout gameplay.

Power-up 1: Super Net

Duration: 8 seconds.

Effect:

- Increases net catching radius.
- Makes catching multiple jellyfish easier.

Power-up 2: Speed Boost

Duration: 7 seconds.

Effect:

- Increases SpongeBob’s movement speed.

Power-up 3: Jellyfish Magnet

Duration: 8 seconds.

Effect:

- Nearby normal jellyfish are attracted toward SpongeBob.

Power-up 4: Bubble Shield

Duration: 6 seconds.

Effect:

- Protects SpongeBob from electric jellyfish.

Power-up 5: Double Points

Duration: 10 seconds.

Effect:

- Doubles all jellyfish-catching scores.

Power-ups should have:

- Unique visual appearances.
- Floating animations.
- Glow effects.
- Collection animations.
- Clear active-duration indicators.

Show the active power-up status on the HUD.

---

9. LEVEL PROGRESSION

Create a proper progression system.

The game should not become repetitive.

Level 1: Beginner Jellyfisher

- Slow jellyfish.
- Fewer electric jellyfish.
- Basic gameplay.
- Easy difficulty.

Level 2: Jellyfish Explorer

- Increased jellyfish spawn rate.
- Faster movement.
- More blue jellyfish.

Level 3: Electric Trouble

- More electric jellyfish.
- More challenging movement patterns.

Level 4: Golden Hunt

- Golden jellyfish appear more frequently.
- Higher score opportunities.

Level 5: Jellyfish Master

- Higher spawn rate.
- Mixed jellyfish types.
- More challenging gameplay.

Difficulty should increase progressively.

Do not make the game unfair.

Each level should introduce a small new challenge.

Include:

- Level completion animation.
- Level transition.
- Level title.
- Progress indicator.
- Score summary.

---

10. GAME HUD — IN-GAME INTERFACE

The HUD must be clean and beautiful.

Display:

TOP LEFT:

- Current Score.
- Current Level.

TOP CENTER:

- Remaining Time.

TOP RIGHT:

- Remaining Lives.
- Pause button.

BOTTOM:

- Active power-ups.
- Catch button.
- Mobile joystick.

HUD design

Use a premium cartoon-inspired interface.

The UI should remain readable against the underwater background.

Use subtle translucent panels where appropriate.

Do not obstruct the gameplay area.

Animations should be subtle and quick.

Score updates should have a small pop animation.

When a player catches a golden jellyfish, trigger a more noticeable celebration effect.

---

11. GAME MODES

Implement the following modes.

A. CLASSIC MODE

Main game mode.

- 90-second timer.
- Catch as many jellyfish as possible.
- Avoid electric jellyfish.
- Collect power-ups.
- Beat the high score.

B. TIME ATTACK

- 60-second timer.
- Faster gameplay.
- Higher jellyfish spawn rate.
- Focus on maximum score.

C. RELAX MODE

- No timer.
- No lives.
- No game over.
- Relaxing underwater experience.
- Catch jellyfish freely.

If development scope becomes too large, prioritize a fully polished Classic Mode first and implement the other modes only after the core gameplay is complete.

---

12. CHARACTERS & RANDOM EVENTS

Add recognizable SpongeBob universe characters as occasional environmental interactions.

Patrick Star

Patrick can randomly appear.

Possible interactions:

- Waves at SpongeBob.
- Accidentally scares jellyfish.
- Occasionally drops a bonus item.
- Appears in funny random events.

Squidward

Squidward can occasionally appear in the background.

He may show an annoyed expression when SpongeBob catches too many jellyfish.

Sandy Cheeks

Sandy can occasionally provide a bonus power-up.

Plankton

Plankton can introduce special challenge events.

These interactions should be occasional and should not interrupt normal gameplay.

Character appearances should have proper animations.

Do not create random characters that do not belong in the SpongeBob universe.

Use properly licensed or user-provided character assets where necessary.

---

13. ANIMATION & PHYSICS ENGINE

THIS IS ONE OF THE MOST IMPORTANT PARTS OF THE PROJECT.

I want the gameplay to feel extremely smooth.

Animation quality is a major priority.

Character animation

Implement:

- Idle animation.
- Movement animation.
- Catching animation.
- Shock animation.
- Victory animation.
- Game-over reaction.

Environment animation

Implement:

- Floating bubbles.
- Moving sea plants.
- Floating jellyfish.
- Background particle movement.
- Subtle underwater light rays.
- Floating dust particles.
- Ambient environmental motion.

UI animation

Implement:

- Button hover effects.
- Button press effects.

- Menu transitions.
- Score pop animations.
- Level transition animations.
- Modal animations.
- Power-up activation animations.

Physics

Use smooth interpolation and appropriate easing.

Use requestAnimationFrame for rendering.

Use delta time for movement and animation calculations.

Avoid unnecessary React re-renders during gameplay.

Keep high-frequency game updates inside the game engine rather than triggering a React render on every frame.

Performance targets

- Aim for 60 FPS on supported devices.
- Avoid unnecessary memory allocation inside the game loop.
- Optimize asset loading.
- Avoid excessive DOM elements during gameplay.
- Use efficient collision detection.
- Pause or reduce background animation when the tab is inactive.

The game should feel responsive on mid-range mobile devices.

Do not sacrifice gameplay performance for unnecessary visual effects.

---

14. SOUND DESIGN

Sound is important for immersion.

Include:

- Underwater ambient background sound.
- Bubble sound effects.
- Jellyfish catching sound.
- Golden jellyfish bonus sound.
- Power-up activation sound.
- Electric shock sound.
- Button interaction sounds.
- Level completion sound.
- Game-over sound.

Use original or appropriately licensed audio assets.

If sound assets are unavailable, create simple synthesized effects using the Web Audio API.

Music

Use a cheerful, playful, relaxing underwater-inspired background soundtrack.

Provide:

- Music ON/OFF.
- Sound Effects ON/OFF.
- Volume control.

Do not autoplay audio before user interaction.

---

15. PAUSE MENU

When the player presses pause:

Show a beautiful overlay containing:

- Resume.
- Restart.
- Settings.
- Main Menu.

Pause must completely stop gameplay timers and movement.

When the game resumes, continue from the correct state.

---

16. GAME OVER SCREEN

When the timer reaches zero or the player loses all lives:

Show a polished game-over screen.

Include:

- SpongeBob’s reaction.
- Final Score.
- Best Score.
- Jellyfish Caught.
- Golden Jellyfish Caught.
- Accuracy.
- Level Reached.

Buttons:

- PLAY AGAIN.
- MAIN MENU.

Add a satisfying score-counting animation.

If the player beats their previous high score, display:

NEW HIGH SCORE!

Make this moment feel rewarding.

---

17. LOCAL STORAGE & PLAYER PROGRESSION

Use browser LocalStorage to save:

- Best Score.
- Total Jellyfish Caught.
- Highest Level Reached.
- Total Games Played.
- Unlocked Game Modes.
- Sound Preferences.
- Music Preferences.

The game should remember progress after refreshing the browser.

No backend is necessary for the initial version.

Do not pretend that local scores are a global online leaderboard.

---

18. SETTINGS & ACCESSIBILITY

Create a dedicated settings screen.

Include:

- Music volume.
- Sound effects volume.
- Music toggle.
- Sound toggle.
- Animation intensity option.
- Control sensitivity.
- Mobile control size.

Also support:

- Keyboard navigation in menus.
- Clear button labels.
- Readable contrast.
- Reduced-motion preference where practical.

---

19. LOADING EXPERIENCE

Create a beautiful loading screen.

Show:

- Underwater background.

- Floating bubbles.
- SpongeBob illustration.
- Loading progress indicator.
- Playful loading messages.

Examples:

“Preparing Jellyfish Fields…”

“Looking for Patrick…”

“Getting the jellyfishing net ready…”

Loading should only remain visible while actual resources are being prepared.

Do not use fake loading delays.

---

20. RESPONSIVE DESIGN

The game must be fully responsive.

Support:

- iPhone.
- Android phones.
- iPad.
- Android tablets.
- Laptop.
- Desktop.

Mobile-first gameplay is essential.

On mobile:

- Use a dedicated touch-control layout.
- Keep the game area appropriately sized.
- Respect safe areas and device notches.
- Prevent unwanted page scrolling during gameplay.
- Ensure buttons are large enough for fingers.

On desktop:

- Use the full available game area.
- Provide keyboard controls.
- Keep the HUD properly positioned.

Test portrait and landscape orientations.

If portrait gameplay becomes too constrained, provide an optional landscape recommendation rather than forcing an inconvenient layout.

---

21. TECHNOLOGY STACK

Use a modern, maintainable frontend architecture.

Preferred stack:

- React.
- TypeScript.
- Vite.
- HTML5 Canvas for the gameplay engine.
- CSS3 for UI and transitions.

- Framer Motion where useful for menu and UI animations.
- Web Audio API for sound.
- LocalStorage for persistence.

You may use lightweight libraries when they provide genuine value.

Avoid unnecessary dependencies.

Do not use a heavy game engine unless there is a strong technical reason.

Suggested architecture

src/

components/

MainMenu/

GameHUD/

MobileControls/

Settings/

PauseMenu/

GameOver/

LoadingScreen/

game/

GameEngine.ts

Player.ts

Jellyfish.ts

PowerUp.ts

CollisionSystem.ts

SpawnManager.ts

LevelManager.ts

InputManager.ts

AudioManager.ts

hooks/

useGame.ts

assets/

characters/

backgrounds/

jellyfish/

ui/

sounds/

styles/

global.css

animations.css

responsive.css

utils/

storage.ts

constants.ts

App.tsx

main.tsx

You may adjust the structure if needed, but maintain clear separation between the game engine, UI, assets, and data.

---

22. VISUAL ASSET MANAGEMENT

Do not build the entire visual experience using random emoji characters.

Use consistent visual assets.

Prioritize:

- Character sprites.
- Background illustrations.
- Jellyfish sprites.
- Net graphics.
- Environmental elements.
- Particle effects.

If actual visual assets are unavailable, create original vector illustrations, CSS-based elements, or procedural Canvas graphics.

The game must still look cohesive and visually polished without external image dependencies.

Organize assets properly.

Avoid broken image links.

Do not hotlink images from random websites.

---

23. WEBSITE EXPERIENCE

The game should open directly into the main menu.

I do not want a traditional marketing website.

The website itself IS the game.

However, include a small, tasteful footer containing:

- About the Game.
- Credits.
- Fan-made project disclaimer.
- GitHub repository link placeholder.

Keep the footer secondary.

The game should occupy almost the entire screen.

Do not add unnecessary sections such as testimonials, pricing, marketing banners, or subscription cards.

---

24. DEPLOYMENT REQUIREMENTS

The final project must be ready for deployment through:

- Vercel.
- Netlify.
- GitHub Pages, if compatible with the selected configuration.

Use a production-ready build configuration.

Requirements:

- npm install must work.
- npm run dev must work.
- npm run build must work.
- npm run preview should work.
- No broken routes.
- No missing assets.
- No console errors.
- No TypeScript errors.
- Proper relative asset paths.
- Correct SPA fallback configuration where needed.

Include:

- README.md.
- Setup instructions.
- Development instructions.
- Build instructions.
- Deployment instructions.
- Controls documentation.
- Credits and asset licensing information.

---

25. DEVELOPMENT WORKFLOW

Follow this development process.

PHASE 1 — FOUNDATION

- Set up the project.
- Establish the visual design system.
- Create the responsive application shell.
- Implement the main menu.
- Establish the game engine architecture.

PHASE 2 — CORE GAMEPLAY

- Implement SpongeBob movement.
- Implement jellyfish spawning.

- Implement catching mechanics.
- Implement collision detection.
- Implement scoring.
- Implement timer and lives.

PHASE 3 — ANIMATIONS

- Implement character animations.
- Implement jellyfish movement.
- Implement environmental effects.
- Implement UI transitions.
- Implement particle effects.

PHASE 4 — GAME SYSTEMS

- Implement power-ups.
- Implement levels.
- Implement game modes.
- Implement random character events.
- Implement pause and game-over screens.

PHASE 5 — AUDIO & PERSISTENCE

- Implement sound effects.
- Implement background audio.
- Implement settings.

- Implement LocalStorage.

PHASE 6 — OPTIMIZATION & TESTING

- Test desktop controls.
- Test touch controls.
- Test mobile responsiveness.
- Test collision detection.
- Test scoring.
- Test game progression.
- Test pause/resume.
- Test game restart.
- Test persistence.
- Test production build.

Do not skip testing.

---

26. STRICT QUALITY REQUIREMENTS

These requirements are non-negotiable.

1. Build a real playable game, not a static mockup.
2. The player must be able to move SpongeBob smoothly.
3. Jellyfish must move independently.
4. Catching must work through actual collision detection.
5. Scores must update correctly.

6. Different jellyfish must have different scores and behaviors.
7. Electric jellyfish must cause damage.
8. Power-ups must have functional effects and durations.
9. Levels must progress correctly.
10. Pause and resume must work properly.
11. Game-over and restart must work properly.
12. Desktop keyboard controls must work.
13. Mobile touchscreen controls must work.
14. Game progress must save correctly.
15. The game must work after production deployment.
16. The UI must remain consistent across all screens.
17. Animations must not interfere with controls or gameplay.
18. Avoid placeholder buttons that do nothing.
19. Avoid fake functionality.
20. Do not leave unfinished sections while claiming the game is complete.

If a feature is too ambitious, implement a simpler but fully functional version instead of creating a non-working placeholder.

---

27. FINAL CREATIVE DIRECTION

I want someone opening this game to immediately feel:

“This is so cute!”

“This looks surprisingly premium.”

“This actually feels like SpongeBob.”

“I want to play another round.”

The game should create nostalgia while still feeling modern.

Prioritize:

- Visual consistency.
- Smooth gameplay.
- Beautiful animation.
- Responsive controls.
- Cute character interactions.
- Satisfying feedback.
- Performance.
- Simplicity.

Remember:

A small game executed exceptionally well is better than a huge game executed poorly.

Do not overcomplicate the design.

Do not sacrifice functionality for aesthetics.

Do not sacrifice performance for animation.

Balance all three.

---

28. FINAL INSTRUCTION

Now begin building the complete project.

First, inspect the development environment and available project files.

Then establish the architecture and implement the actual game.

Do not stop after creating a plan.

Do not just explain what you would build.

Write the actual code, create the necessary files, implement the features, and verify that the application builds successfully.

If you have access to a live preview, use it to inspect the application.

Fix visible UI issues, broken interactions, and gameplay bugs before declaring completion.

The final result must be a complete, responsive, smooth, playable SpongeBob: Jellyfish Catcher browser game that can be deployed to Vercel or Netlify.

Build something delightful. Make every interaction feel intentional. Make the gameplay feel alive.

আমার কিছু গুরুত্বপূর্ণ পরামর্শ

ব্রো, Prompt দেওয়ার পাশাপাশি কয়েকটা জিনিস মাথায় রাখবি।

১. **Google AI Studio-**তে কীভাবে কাজ করবি**?**

একবারে পুরো prompt দিয়ে দিস। এরপর AI-কে কাজ করতে দে।

প্রথমবারেই ১০০% perfect না হলে সমস্যা নেই। Development-এর সময় ছোট ছোট prompt দিয়ে refine করবি।

২. **Apple-**এর মতো **UI** বলতে কী বুঝিয়েছি**?**

আমি Apple-এর কোনো নির্দিষ্ট UI কপি করতে বলিনি। বরং তাদের design philosophy অনুসরণ করতে বলেছি:

- Clean interface
- Perfect spacing
- Smooth transitions
- Consistent typography
- Minimal visual clutter
- Beautiful micro-interactions

৩. Hosting

আমার recommendation:

Platform	Recommendation
Vercel	⭐ Best choice
Netlify	Excellent
GitHub Pages	Good for static hosting

তুই Vercel-এ GitHub repository connect করলেই পরবর্তীতে GitHub-এ নতুন code push করলে automatic redeployment করতে পারবি।

৪. একটা জিনিস অবশ্যই করবি

SpongeBob-এর original character artwork, background এবং cartoon scenes ব্যবহার করতে হলে licensed বা অনুমোদিত assets ব্যবহার করবি। শুধু AI-কে character-এর নাম বললে সবসময় recognizable ও consistent character artwork তৈরি হবে না।

আমার final recommendation: প্রথমে একটা অসাধারণ Classic Mode তৈরি কর। সেটার movement, catching animation, jellyfish physics এবং touch controls perfect কর। এরপর বাকি features যোগ করবি।

কারণ এই game-এর আসল মজা হবে SpongeBob-কে control করার অনুভূতিতে। সেটা smooth হলে পুরো project-টাই অন্য লেভেলে চলে যাবে। 🧽💛